1.What is destructor? List its characteristics. Explain the use of default copy constructor with an appropriate example.

Solution

Destructors are members functions in a class that delete an object. They are called when the class object goes out of scope such as when the function ends, the program ends, a delete variable is called etc.

Destructors are different from normal member functions as they don't take any argument and don't return anything. Also, destructors have the same name as their class and their name is preceded by a tilde(~).

The syntax for defining the destructor within the class:

```
~ <class-name>() {
// some instructions
}.
// C++ program to demonstrate the execution of constructor
// and destructor
#include <iostream>
using namespace std;
class Test {
public:
// User-Defined Constructor
Test() { cout << "\n Constructor executed"; }
// User-Defined Destructor
~Test() { cout << "\nDestructor executed"; }
};
main()
{
Test t;
return 0;
}
```

Characteristics of a Destructor

- A destructor is also a special member function like a constructor. Destructor destroys the class objects created by the constructor.
- Destructor has the same name as their class name preceded by a tilde (~) symbol.
- It is not possible to define more than one destructor.
- The destructor is only one way to destroy the object created by the constructor. Hence, destructor cannot be overloaded.
- It cannot be declared static or const.
- Destructor neither requires any argument nor returns any value.

A default copy constructor in C++ is a constructor that initializes a new object as a copy of an existing object of the same class.

- Default copy constructor is needed if we don't define our own copy constructor, in the C++ for each class which works fine in general.
- They are used to initialize member objects.

```
// Implicit copy constructor Calling

#include <iostream>

using namespace std;

class Sample {

int id;

public:

void init(int x) { id = x; }

void display() { cout << endl << "ID=" << id; }

};

int main()

{

Sample obj1;

obj1.init(10);

obj1.display();
```

```
// Implicit Copy Constructor Calling
Sample obj2(obj1); // or obj2=obj1;
obj2.display();
return 0;
}
```

2. Write a program to create a class named height with data members meter(int) and centimeter(int). Overload the binary + operator using friend function to add two heights.

Solution

```
#include <iostream>
using namespace std;
class Height {
public:
int meter, centimeter;
// Default constructor
Height() {
this->meter = 0;
this->centimeter = 0;
}
// Parameterized constructor
Height(int m, int cm) {
this->meter = m;
this->centimeter = cm;
}
// Friend function to overload + operator
// This is called by value
friend Height operator+(Height h1, Height h2) {
Height result;
```

```
// Add meters and centimeters separately
result.meter = h1.meter + h2.meter;
result.centimeter = h1.centimeter + h2.centimeter;
// If the centimeters exceed 100, convert to meters
if (result.centimeter >= 100) {
result.meter += result.centimeter / 100;
result.centimeter = result.centimeter % 100;
}
return result;
}
};
// Driver Code
int main() {
Height h1(2, 75); // 2 meters 75 centimeters
Height h2(3, 50); // 3 meters 50 centimeters
Height h3;
// Use overloaded operator
h3 = h1 + h2;
cout << "\nTotal Height: " << h3.meter << " meters and " << h3.centimeter << " centimeters." << endl;
return 0;
}
```

3. Define object. What are the benefits of Object Oriented Programming Language?

Solution

An object is an instance of a class. They are allocated memory space whenever they are created. An object is created many times as per requirement. It can be manipulated. An object is a physical entity. Objects are like a variable of the class. Each object has its own values, which are associated with it.

Syntax: Instantiating an object for a Class in C++ is as follows:

```
class Student {
public:
void put(){
cout<<"Function Called"<<endl;
}
}; // The class is declared here
int main(){
Student s1; // Object created
s1.put();
}
```

The benefits of Object Oriented Programming Language are:

- OOP language allows to break the program into the bit-sized problems that can be solved easily (one object at a time).
- The new technology promises greater programmer productivity, better quality of software and lesser maintenance cost.
- OOP systems can be easily upgraded from small to large systems.
- It is possible that multiple instances of objects co-exist without any interference,
- It is very easy to partition the work in a project based on objects.
- By using inheritance, we can eliminate redundant code and extend the use of existing classes.
- The principle of data hiding helps the programmer to build secure programs which cannot be invaded by the code in other parts of the program.

4. Explain the significance of type conversion. How do we achieve dynamic memory allocation in C++? Explain with an example.

Solution

Type casting, also known as type conversion, is an important concept that allows us to convert one data type into another. It is useful when we need to perform operations on different data types or when you want to store a value of one data type into a variable of another data type.

Type casting is the process of converting the value of a single data type into another data type. This conversion is done either automatically or manually. The compiler performs the automatic conversion, and a programmer does the manual conversion. To use a variable in a particular way in automatic conversion, we need to explicitly tell the Java compiler to convert a variable from one data type to another data type.

Dynamic memory allocation in C++ refers to performing memory allocation manually by a programmer. C++ has two operators new, that perform the task of allocating the memory in a better and easier way.

The new operator denotes a request for memory allocation. If sufficient memory is available, a new operator initializes the memory and returns the address of the newly allocated and initialized memory to the pointer variable.

Syntax to use new operator

```
pointer-variable = new data-type;
```

Example of new operator

```
using namespace std;
int main()
{
// pointer to store the address returned by the new
int* ptr;
// allocating memory for integer
ptr = new int;
// assigning value using dereference operator
*ptr = 10;
// printing value and address
cout << "Address: " << ptr << endl;
cout << "Value: " << *ptr;
return 0;
}
```

5. [What is the benefit of passing object as arguments? Write a program to create a class named actor with data members name and rating. Initialize the data

members and display those names whose rating is greater than 5 using the concept of constant object.

Solution

Passing objects as arguments in C++ offers several benefits that enhance both the flexibility and functionality of the program. Some benefits are:

a. Encapsulation of Data

b. Supports Abstraction

c. Reusability

d. Flexible

e. Polymorphism

```
#include <iostream>

#include <string>

using namespace std;

class Actor {

string name;

int rating;

public:

Actor(string n, int r) {

name = n;

rating = r;

}

int getRating(){

return rating;

}

string getName(){

return name;

}

};
```

```
int main() {
actor1 = Actor("Actor One", 6);
actor2 = Actor("Actor Two", 4);
actor3 = Actor("Actor Three",
if (actor1.getRating() > 5) {
cout << actor1.getName() << " has a rating greater than 5." << endl;
}
if (actor2.getRating() > 5) {
cout << actor2.getName() << " has a rating greater than 5." << endl;
}
if (actor3.getRating() > 5) {
cout << actor3.getName() << " has a rating greater than 5." << endl;
}
return 0;
}
```

6. Create two classesRupee and Dollar respectively. Write conversion operator to convert between Rupee and Dollar assuming that 1 dollar equals 133 rupees. Write a main program that allows the user to enter an amount in either currncy and then converts it to other currency and displays the result.

Solution

```
#include <iostream>
using namespace std;
class Rupee {
private:
float amount;
public:
Rupee(float amt) : amount(amt) {}
operator Dollar();
void display() const {
```

```
cout << amount << " Rupees" << endl;
}
};
class Dollar {
private:
float amount;
public:
Dollar(float amt) : amount(amt) {}
operator Rupee() {
return Rupee(amount * 133);
}
void display() const {
cout << amount << " Dollars" << endl;
}
};
Rupee::operator Dollar() {
return Dollar(amount / 133);
}
int main() {
char choice;
cout << "Enter 'R' for Rupees to Dollars or 'D' for Dollars to Rupees: ";
cin >> choice;
if (choice == 'R' || choice == 'r') {
float amount;
cout << "Enter amount in Rupees: ";
cin >> amount;
Rupee rupee(amount);
Dollar dollar = rupee;
```

```
rupee.display();
dollar.display();
} else if (choice == 'D' || choice == 'd') {
float amount;
cout << "Enter amount in Dollars: ";
cin >> amount;
Dollar dollar(amount);
Rupee rupee = dollar;
dollar.display();
rupee.display();
} else {
cout << "Invalid choice! Please enter 'R' or 'D'." << endl;
}
return 0;
}
```

7. Explain the practical implication of protected specifier in inheritance. list advantages and disadvantages of inheritance.

Solution

practical implications of using the protected specifier in inheritance:

a. Members declared as protected in a parent class are accessible within the subclass. This allows subclasses to use and modify these members, enabling them to inherit common behavior and state from the parent class.

b. It allows subclasses to have access to critical information and functionality without exposing these members to the world outside the class hierarchy.

c. Subclasses can directly access and utilize the protected members to implement specific behaviors without needing to redefine or duplicate the code in the parent class.

d. Using protected can lead to better design practices by encouraging developers to think about which members should be part of the class interface and which should be hidden from the outside world.

e. While protected members can be beneficial, overusing them can lead to complex class hierarchies and tight coupling between classes.

The advantages and disadvantages of inheritance in object-oriented programming are as followed.

Advantage:

a. Inheritance allows classes to reuse code from existing classes, reducing redundancy and saving development time.

b. Subclasses can override methods of the parent class to provide specific implementations, allowing for more customized behavior.

c. It helps in organizing classes in a hierarchical manner, making the code easier to understand and maintain.

d. Inheritance supports polymorphism, allowing objects of different classes to be treated as objects of a common superclass, which enhances flexibility.

e. Inheritance provides a clear structure for the program, making it easier to navigate and manage.

Disadvantage:

a. A deep inheritance hierarchy can lead to complexity, making the code harder to understand and maintain.

b. Inheritance can lead to a rigid structure where classes are forced to adhere to the hierarchy, limiting design flexibility.

c. In some cases, inheritance can introduce performance overhead due to the additional method resolution and look-up time.

d. In some cases, using composition might be more appropriate than inheritance, leading to better flexibility and maintainability.

8. Explain function template overloading with suitable example.

Solution

Function template overloading allows us to define multiple function templates with the same name but different parameter lists. This is similar to regular function overloading, where multiple functions with the same name but different signatures can coexist.

Function templates can be overloaded just like normal functions, based on the number or type of parameters. The compiler selects the appropriate template

based on the arguments passed to the function call. You can have both normal function overloads and template overloads with the same name, and the most appropriate version is chosen based on the arguments.

Example of Function template overloading :

```
#include <iostream>

using namespace std;

// Template function for adding two same-type variables

template <typename T>

T add(T a, T b) {

cout << "Adding two variables of the same type: ";

return a + b;

}

// Template function for adding two different types of variables

template <typename T1, typename T2>

auto add(T1 a, T2 b) -> decltype(a + b) {

cout << "Adding two variables of different types: ";

return a + b;

}

int main() {

// Calling the same-type template

int x = 10, y = 20;

cout << add(x, y) << endl; // Matches the first template

// Calling the different-type template

int a = 5;

double b = 4.5;

cout << add(a, b) << endl; // Matches the second template

return 0;

}
```

9. Write a program that stores information of employees in a file and displays the file's content in ascending order of their salary.

Solution

```
#include <iostream>

#include <fstream>

#include <string>

using namespace std;

// Structure to store employee data

struct Employee {

string name;

int id;

float salary;

};

int main() {

fstream file;

Employee emp[100]; // Array to store multiple employees

int n;

// Write to file

file.open("employees.txt", ios::out);

if (!file) {

cout << "Error in opening file!" << endl;

return 0;

}

cout << "Enter the number of employees: ";

cin >> n;

for (int i = 0; i < n; i++) {

cout << "Enter name of employee " << i + 1 << ": ";

cin >> emp[i].name;
```

```
cout << "Enter ID of employee " << i + 1 << ": ";
cin >> emp[i].id;
cout << "Enter salary of employee " << i + 1 << ": ";
cin >> emp[i].salary;
// Write employee data to the file
file << emp[i].name << " " << emp[i].id << " " << emp[i].salary << endl;
}
file.close();
// Read from file and display sorted by salary
file.open("employees.txt", ios::in);
if (!file) {
cout << "Error in opening file!" << endl;
return 0;
}
// Reading employee data back from the file
int i = 0;
while (file >> emp[i].name >> emp[i].id >> emp[i].salary) {
i++;
}
file.close();
// Sort employees by salary
for (int j = 0; j < i - 1; j++) {
for (int k = j + 1; k < i; k++) {
if (emp[j].salary > emp[k].salary) {
Employee temp = emp[j];
emp[j] = emp[k];
emp[k] = temp;
}
```

```
}
}
// Display sorted employee data
cout << "\nEmployees sorted by salary:\n";
for (int j = 0; j < i; j++) {
cout << "Name: " << emp[j].name << ", ID: " << emp[j].id << ", Salary: " << emp[j].salary << endl;
}
return 0;
}
```

10. Write short notes on :

    a. This pointer

    b. ios member function

Solution

a. this pointer :
   "this pointer" is a special pointer available within all non-static member functions of a class in C++. It points to the object for which the member function is called. It is automatically passed to all non-static member functions and does not need to be declared explicitly. Inside a member function, it refers to the address of the object that invoked the function, allowing access to the object's data members and member functions.
   It is used to distinguish between local variables and class data members when they have the same name. It is useful in method chaining, where multiple calls are linked together. It is a fundamental feature of C++ that helps in accessing the members of an object, especially when there is ambiguity, and enables method chaining for cleaner, more expressive code.

b. ios member function :
   The ios member functions in C++ are part of the Input/Output Stream (I/O stream) class hierarchy, which includes classes like iostream, ifstream, ofstream, and others. The class ios is the base class that provides common functionality to input and output streams.

11. Difference between compile time and run time polymorphism.

Compile Time Polymorphism	Run time Polymorphism
In Compile time Polymorphism, the call is resolved by the compiler.	In Run time Polymorphism, the call is not resolved by the compiler.
It is also known as Static binding, Early binding and overloading as well.	It is also known as Dynamic binding, Late binding and overriding as well.
It is achieved by method overloading.	It is achieved by virtual functions and pointers.
It provides fast execution because the method that needs to be executed is known early at the compile time.	It provides slow execution as compare to early binding because the method that needs to be executed is known at the runtime.
Compile time polymorphism is less flexible as all things execute at compile time.	Run time polymorphism is more flexible as all things execute at run time.

12. When inline functions may not work? Define and write syntax for default arguments. Write a program to display N number of characters by using default arguments for both parameters. Assume that the function takes two arguments, one character to be printed and other how many times the character to be printed respectively.

Solution

Inline functions may not work in certain situations, such as:

1. Complex Functions: Inline functions are expanded by the compiler at the call site, so if a function is too complex or contains a lot of code, it may result in code bloat and decrease in performance rather than improvement. In such cases, the compiler may choose not to inline the function.
2. Recursive Functions: Recursive functions, which call themselves, are typically not suitable for inlining. Inlining them would lead to infinite expansion.
3. Functions with Loops: Functions that contain loops may not be good candidates for inlining because they can lead to code duplication and increased binary size.

Here's the syntax for default arguments in C++:

return_type function_name(parameter_type parameter_name = default_value);

Now, let's write a C++ program that displays a character a certain number of times using default arguments:

```
#include <iostream>

using namespace std;

// Function to display a character 'ch' 'n' number of times with default arguments

void displayCharacter(char ch = '*', int n = 10) {

    for (int i = 0; i < n; i++) {

        cout << ch;

    }

    cout << endl;

}

int main() {

    int num;

    char inputChar;

    // Display '*' 10 times (using default arguments)

    displayCharacter();

    // Display '#' 5 times

    displayCharacter('#', 5);

    // Prompt the user for input

    cout << "Enter a character: ";

    cin >> inputChar;

    cout << "Enter the number of times to display the character: ";

    cin >> num;

    // Display the user-defined character 'num' times

    displayCharacter(inputChar, num);

    return 0;
```

}In this program, we have defined a function displayCharacter with default arguments char ch = '*' and int n = 10. This means that if you call the function without providing any arguments, it will display '*' 10 times by default. You can also provide custom values for the character and the number of times it should be displayed when calling the function.

13. What is constructor? Why constructor is needed in a class? Illustrate the types of constructor with an example.

Solution

A constructor in object-oriented programming is a special member function of a class that gets called automatically when an object of the class is created. It is used to initialize the object's data members and perform any necessary setup or initialization tasks. Constructors have the same name as the class and do not have a return type.

Constructors are needed in a class for several reasons:

1. Initialization: Constructors ensure that objects of a class are properly initialized when they are created. This is important to avoid undefined or unexpected behavior caused by uninitialized data members.
2. Setting Default Values: Constructors can set default values for data members if no explicit values are provided during object creation. This allows for flexibility in object initialization.
3. Resource Allocation: Constructors can allocate resources like memory, file handles, network connections, or database connections. This ensures that the necessary resources are acquired and managed properly.
4. Parameterization: Constructors can accept parameters to customize the initialization of objects. This allows you to create objects with different initial states based on the provided arguments.

Types of Constructors:

1. Default Constructor: A default constructor is a constructor that takes no parameters. If you don't provide any constructors in your class, the compiler will automatically generate a default constructor for you. It initializes the data members with default values (zero or null) or performs no initialization if possible.

```
class MyClass {
public:
```

```
// Default constructor (implicitly provided by the compiler)
};
```

2. Parameterized Constructor: A parameterized constructor is a constructor that takes one or more parameters. It allows you to initialize the object's data members with specific values provided during object creation.

```
class Student {
private:
int rollNumber;
std::string name;

public:
// Parameterized constructor
Student(int roll, const std::string& n) : rollNumber(roll), name(n) {}
};

int main() {
// Creating an object of Student with parameterized constructor
Student student1(101, "Alice");
}
```

3. Copy Constructor: A copy constructor is a constructor that creates a new object by copying the values from an existing object of the same class. It is used when objects are passed by value or when objects are created as copies of existing objects.

```
class Point {
private:
int x, y;

public:
// Copy constructor
Point(const Point& other) : x(other.x), y(other.y) {}
};

int main() {
Point p1(1, 2);
Point p2 = p1; // Using the copy constructor to create p2 as a copy of p1
}
```

14. Write a C++ program to display the number of objects created using static member.

Solution

```
#include <iostream>
using namespace std;
class MyClass {
private:
    static int count; // Static member variable to keep track of the object count
public:
    MyClass() {
        count++; // Increment the object count when a new object is created
    }
    // Static member function to retrieve the object count
    static int getObjectCount() {
        return count;
    }
};
// Initialize the static member variable to 0
int MyClass::count = 0;
int main() {
    MyClass obj1; // Create the first object
    MyClass obj2; // Create the second object
    MyClass obj3; // Create the third object
    // Display the number of objects created
    cout << "Number of objects created: " << MyClass::getObjectCount() << endl;
    return 0;
}
```

15. Explain about friend function and friend class with an example.

Solution

In C++, a friend function and a friend class are features that provide access to the private and protected members of a class from outside the class. This allows certain functions or classes to have special access privileges, even though they are not part of the class hierarchy. Friends are typically used in situations where you need to grant specific functions or classes access to the private members of a class while still maintaining encapsulation.

Friend Function:

A friend function is a function that is not a member of a class but is granted access to the private and protected members of the class. To declare a function as a friend of a class, you use the friend keyword in the class declaration.

Here's an example:

```
#include <iostream>

using namespace std;

class MyClass {

private:

    int data;

public:

    MyClass(int value) : data(value) {}

    // Declare a friend function

    friend void displayData(const MyClass& obj);

};

// Define the friend function

void displayData(const MyClass& obj) {

    cout << "Data from friend function: " << obj.data << endl;

}

int main() {

    MyClass obj(42);

    displayData(obj); // Call the friend function to access private data

    return 0;

}
```

In this example, the displayData function is declared as a friend of the MyClass class. It can access the private member data of MyClass objects.

Friend Class:

A friend class is a class that is granted access to the private and protected members of another class. To declare a class as a friend of another class, you use the friend keyword in the class declaration.

Here's an example:

```
#include <iostream>

using namespace std;

class MyClass {

private:

    int data;

public:

    MyClass(int value) : data(value) {}

    // Declare a friend class

    friend class FriendClass;

};

// Define the friend class

class FriendClass {

public:

    void displayData(const MyClass& obj) {

        cout << "Data from friend class: " << obj.data << endl;

    }

};

int main() {

    MyClass obj(42);

    FriendClass fc;

    fc.displayData(obj); // Call the friend class method to access private data

    return 0;
```

}

In this example, the FriendClass class is declared as a friend of the MyClass class. Objects of FriendClass can access the private member data of MyClass objects.

16. What is the use of reinterpret cast operator? Why do we need virtual function?

Solution

Use of reinterpret_cast Operator:

In C++, the `reinterpret_cast` operator is used for type casting or converting one pointer type to another pointer type, even if the two pointer types are unrelated. It is the most powerful, yet potentially dangerous, casting operator because it allows you to perform low-level type conversions that can lead to undefined behavior if not used carefully. The `reinterpret_cast` operator should be used sparingly and with a deep understanding of the underlying memory layout of the objects involved.

Here are some common use cases for the `reinterpret_cast` operator:

– Pointer Type Conversion: It can be used to convert between different pointer types, such as converting a pointer to an integer or vice versa.

```
int num = 42;
int* ptr = &num;
uintptr_t addr = reinterpret_cast<uintptr_t>(ptr);
```

– Type Punning: It is used in situations where you need to interpret the bits of one data type as another data type. This is commonly used in low-level programming and systems programming.

```
float floatValue = 3.14f;
int* intPtr = reinterpret_cast<int*>(&floatValue);
```

– Working with Low-Level Memory: When working with raw memory buffers or hardware interactions, `reinterpret_cast` can be used to interpret the memory content as different types.

```
struct HardwareRegister {
uint32_t value;
};

HardwareRegister* reg = reinterpret_cast<HardwareRegister*>(0x12345678);
```

It’s important to exercise caution when using `reinterpret_cast`, as it bypasses many of the type safety checks provided by C++ and can lead to undefined behavior if misused.

Need for Virtual Functions:

Virtual functions in C++ are a fundamental feature of polymorphism and play a crucial role in building flexible and extensible object-oriented programs. They are used in scenarios where you need to perform different operations on objects of different derived classes through a common base class pointer. Virtual functions enable runtime method binding, which means the appropriate function implementation is determined at runtime based on the actual derived class type of the object being operated on.

Here's why we need virtual functions:

– Polymorphism: Virtual functions enable polymorphism, allowing you to write code that can work with objects of various derived classes through a common interface (base class pointer or reference).

```
class Shape {
public:
virtual void draw() {
// Base class draw function
}
};

class Circle : public Shape {
public:
void draw() override {
// Circle-specific draw function
}
};

void drawShape(Shape* shapePtr) {
shapePtr->draw(); // Calls the appropriate derived class's draw function
}
```

– Dynamic Binding: Virtual functions provide dynamic binding, which means the correct function implementation is determined at runtime based on the actual type of the object, not just the type of the pointer or reference.

– Extensibility: Virtual functions allow you to add new derived classes without modifying existing code that works with the base class. This promotes code extensibility and flexibility.

– Abstraction: They enable the creation of abstract base classes, which define a common interface for derived classes to implement. This abstraction simplifies complex systems and promotes code organization.

17. Differentiate between concrete class and abstract class. Define class template and function template with syntax.

Solution

Concrete Class:

A concrete class in C++ is a class that can be instantiated, meaning you can create objects (instances) of that class. Concrete classes define and provide implementations for all their member functions. These classes are designed for direct use and are used to create objects with specific behaviors and attributes. Concrete classes cannot have pure virtual functions, and they provide complete implementations for all their member functions.

Here's an example of a concrete class:

```
class Rectangle {
private:
int length;
int width;

public:
Rectangle(int len, int wid) : length(len), width(wid) {}

int calculateArea() {
return length * width;
}
};
```

Abstract Class:

An abstract class in C++ is a class that cannot be instantiated on its own. It serves as a base class or interface for other classes and contains one or more pure virtual functions. A pure virtual function is a function declared in the abstract class but without a function body, making it mandatory for derived classes to provide their implementations for these functions. Abstract classes are used to define a common interface that derived classes must adhere to.

Here's an example of an abstract class:

```
class Shape {
public:
virtual double calculateArea() = 0; // Pure virtual function

virtual void draw() {
// Virtual function with default implementation
}
};
```

Class Template:

A class template in C++ is a blueprint for creating a family of classes. It allows you to define a class in which some or all of its data members and member functions are parameterized by one or more template parameters. These parameters are specified when you create an instance of the template class, and they allow you to create different classes with the same structure but different data types.

Syntax of a class template:

```
template <typename T1, typename T2, ...>
class ClassName {
// Class definition with template parameters
};
```

Here's an example of a class template for a simple generic container:

```
template <typename T>
class MyContainer {
private:
T data;

public:
MyContainer(T value) : data(value) {}

T getValue() {
return data;
}
};
```

Function Template:

A function template in C++ allows you to define a family of functions where some or all of the function's arguments are parameterized by one or more template parameters. Function templates provide a way to create functions that can work with different data types while maintaining type safety.

Syntax of a function template:

```
template <typename T1, typename T2, ...>
return_type functionName(T1 param1, T2 param2, ...) {
// Function definition with template parameters
}
```

Here's an example of a function template for finding the maximum of two values:

```
template <typename T>
T max(T a, T b) {
return (a > b) ? a : b;
}
```

18. Describe the chain of constructors and destructors in inheritance.

Solution

In C++ inheritance, when you have a base class and one or more derived classes, constructors and destructors are called in a specific order, often referred to as the "chain of constructors" and "chain of destructors." This order ensures that objects are properly constructed and destroyed, preserving the integrity of the inheritance hierarchy. The rules for constructor and destructor chaining are as follows:

Constructor Chaining:

1. Base Class Constructor: When an object of a derived class is created, the constructor of the base class is called first. This ensures that the base class part of the derived object is properly initialized.

2. Derived Class Constructor: After the base class constructor completes its execution, the constructor of the derived class is called. This allows the derived class to initialize its own members.

Here's an example:

```
#include <iostream>

using namespace std;


class Base {

public:

    Base() {

        cout << "Base class constructor" << endl;
```

```
    }
};
class Derived : public Base {
public:
    Derived() {
        cout << "Derived class constructor" << endl;
    }
};
int main() {
    Derived derivedObj;
    return 0;
}
```

When you create a `Derived` object, the output will be:

```
Base class constructor
Derived class constructor
```

Destructor Chaining:

The destruction process follows the reverse order of constructors:

1. Derived Class Destructor: When an object of a derived class goes out of scope or is explicitly deleted, the destructor of the derived class is called first. This allows the derived class to perform its cleanup operations.

2. Base Class Destructor: After the derived class destructor completes its execution, the destructor of the base class is called. This ensures that the base class part of the object is properly cleaned up.

Here's an example:

```
#include <iostream>
using namespace std;
class Base {
public:
    ~Base() {
        cout << "Base class destructor" << endl;
```

```
    }
};
class Derived : public Base {
public:
    ~Derived() {
        cout << "Derived class destructor" << endl;
    }
};
int main() {
    Derived derivedObj;
    return 0;
}
```

When the `derivedObj` goes out of scope or is explicitly destroyed, the output will be:

```
Derived class destructor
Base class destructor
```

In summary, the chain of constructors and destructors in C++ inheritance ensures that the base class part of an object is constructed before the derived class part and is destructed after the derived class part. This order is essential for proper object initialization and cleanup when working with inheritance hierarchies.

19. List any two operators that cannot be overloaded. Write a program to handle the possible exception in taking “age” as input. Here the exception must be thrown in the input is negative or greater than 200.

Solution

Two operators that cannot be overloaded in C++ are:

1. Scope Resolution Operator (::): It is used to access the global scope or to qualify the name of a class, namespace, or enumeration.

2. Conditional Operator (?:): It is used for conditional expressions, and it cannot be overloaded because it is a ternary operator with a fixed behavior.

```
#include <iostream>
#include <stdexcept>
```

```
using namespace std;


int main() {
    int age;
    try {
        cout << "Enter your age: ";
        cin >> age;
        if (age < 0 || age > 200) {
            throw out_of_range("Age is out of range (0-200)");
        }
        cout << "Your age is: " << age << endl;
    } catch (const exception& e) {
        cerr << "Error: " << e.what() << endl;
    }
    return 0;
}
```

20.Create a class named Point with data members x(int) and y(int). Add operator overloading to find the Euclidean distance between two points.

Solution

```
#include <iostream>
#include <cmath>
using namespace std;
class Point {
private:
    int x;
    int y;
public:
    Point(int xCoord, int yCoord) : x(xCoord), y(yCoord) {}
```

```
    // Overload the '-' operator to calculate the Euclidean distance
    double operator-(const Point& other) const {
        int deltaX = x - other.x;  // Use normal minus sign here
        int deltaY = y - other.y;
        return sqrt(deltaX * deltaX + deltaY * deltaY);
    }
};
int main() {
    Point p1(1, 2);
    Point p2(4, 6);
    double distance = p1 - p2;
    cout << "Euclidean distance between p1 and p2: " << distance << endl;
    return 0;
}
```

21. Explain the reason for member function overriding when using virtual function.

Solution

Member function overriding is a key concept in C++ when working with virtual functions and inheritance. The reason for member function overriding is to enable polymorphism, dynamic binding, and the ability to provide specialized implementations of virtual functions in derived classes.

Here's a detailed explanation of why member function overriding is necessary when using virtual functions:

1. Polymorphism:

Polymorphism is the ability of objects of different classes to respond to the same function call in a way that is appropriate for their specific types. In C++, this is achieved through virtual functions and function overriding. By defining a virtual function in a base class and allowing derived classes to provide their own implementations, you enable polymorphism.

2. Dynamic Binding:

Virtual functions enable dynamic binding or late binding. When you call a virtual function on a base class pointer or reference that points to an object of a derived class, the appropriate derived class's function implementation is called at runtime. This is in contrast to static binding, where the function call is resolved at compile-time based on the type of the pointer or reference.

3. Specialization:

Derived classes often have specific behaviors or attributes that are unique to them. By overriding virtual functions, you allow derived classes to provide specialized implementations of those functions to meet their specific requirements. This allows you to take advantage of the base class's interface while tailoring the behavior to the individual-derived classes.

4. Code Reusability and Extensibility:

Virtual functions and member function overriding promote code reusability and extensibility. You can create a base class with a common interface and then derive multiple classes from it, each providing its own implementation of virtual functions. This allows you to write generic code that works with objects of the base class but can also work seamlessly with objects of any derived class.

Here's an example to illustrate the importance of member function overriding with virtual functions:

```
#include <iostream>

using namespace std;

class Shape {

public:

    virtual void draw() {

        cout << "Drawing a shape" << endl;

    }

};

class Circle : public Shape {

public:

    void draw() override {

        cout << "Drawing a circle" << endl;
```

```
    }
};
int main() {
    Shape* shapePtr = new Circle;
    shapePtr->draw(); // Calls the overridden draw function of Circle
    delete shapePtr;
    return 0;
}
```

In this example, the `Shape` class defines a virtual function `draw()`, and the `Circle` class overrides this function with its own specialized implementation. When we create a `Circle` object and call `draw()` on a `Shape*` pointer pointing to that object, the appropriate derived class's function is called at runtime, demonstrating polymorphism and dynamic binding. This is possible because of member function overriding with virtual functions.

22. [What do you mean by stream? Explain different stream class for file input/output.]

Solution

In C++, a stream refers to a sequence of data that flows between a program and a source or destination, such as a file, a keyboard, or a network connection. Streams are used for input and output operations, allowing data to be read from or written to various sources or destinations in a structured and consistent manner. C++ provides a powerful stream library known as the Standard Input/Output Library, which includes a variety of stream classes for file input and output.

Here are the key stream classes for file input/output in C++:

1. ifstream (Input File Stream):

The ifstream class is used for reading data from files. It allows you to open a file for input and perform read operations. You can use `>>` and `getline()` operations to read data from the file.

```
#include <fstream>
ifstream inputFile("data.txt"); // Open a file for reading
```

```
int num;
inputFile >> num; // Read an integer from the file
```

2. ofstream (Output File Stream):

The ofstream class is used for writing data to files. It allows you to open a file for output and perform write operations. You can use `<<` operations to write data to the file.

```
#include <fstream>
ofstream outputFile("output.txt"); // Open a file for writing
int num = 42;
outputFile << num; // Write an integer to the file
```

3. fstream (File Stream):

The fstream class is a combination of `ifstream` and `ofstream` and is used for both reading and writing to files. It allows you to open a file in both input and output modes.

```
#include <fstream>
fstream file("data.txt", ios::in | ios::out); // Open a file for both reading and writing
int num;
file >> num; // Read an integer from the file
num++;
file << num; // Write an updated integer to the file
```

23. What is aggregation? Write a program for implementing following:

Create a class author with attributes name and qualification. Also create a class publication with pname. From these classes derive a classes derive a class book having attributes title and price. Each of the three classes should have getdata() method to get their data from user. The classes should have putdata() method to display the data. Create instance of the class book in main.

Solution

Aggregation in C++ (commonly called as a has-a relationship), is a process in which one class defines a second class as an entity reference. It is a method of reusability of classes. In the simplest possible terms, it is when a class has an object of the other class.

Program Part:

```
#include<iostream>
```

```
using namespace std;
class author {
public:
string name, qualification;
void getdata() {
cout << "\nEnter Author Name : ";
getline(cin, name);
cout << "Enter Author Qualification : ";
getline(cin, qualification);
}
};
class publication {
public:
string pname;
void getdata() {
cout << "\nEnter Publication Name : ";
getline(cin, pname);
}
};
class book: public author, public publication {
public:
string title;
float price;
// Function overriding
void getdata() {
cout << "\nEnter Book Title : ";
getline(cin, title);
cout << "Enter Book price : ";
```

```
cin >> price;
}
void putData(){
cout << "\n\n========== BOOK INFO ==========" << endl;
cout << "\nBook Author : " << name;
cout << "\nBook Author Qualification : " << qualification;
cout << "\nBook Publication : " << pname;
cout << "\nBook Name : " << title;
cout << "\nBook Price : Rs." << price;
}
};
int main() {
// Created an object of class book
book obj;
//read the data each class by scope resolution
obj.author::getdata();
obj.publication::getdata();
obj.getdata();
//display result
obj.putData();
return 0;
}
```

The output of above program is

```
Enter Author Name : SURESH CHAND
Enter Author Qualification : BSC. CSIT
Enter Publication Name : HAMROCSIT
Enter Book Title : C++ PROGRAMMING
Enter Book price : 100
```

========== BOOK INFO ==========

Book Author : SURESH CHAND

Book Author Qualification : BSC. CSIT

Book Publication : HAMROCSIT

Book Name : C++ PROGRAMMING

Book Price : Rs.100

24. What is operator overloading? Why it is necessary to overload an operator? Write a program for overloading comparision operator.

Solution

The method of making operators to work for user defined classes and having the ability to provide operators with a special user defined meaning is known as operator overloading.

In the other words, Operator overloading is a type of polymorphism in which an operator is overloaded to give user defined meaning to it.

For Example:

'+' operator can be overloaded to perform on various data type like addition for integer, concatenation for string etc.

Need of Operator Overloading in c++:

It makes the program more readable and easier to understand by allowing manipulation of objects of class's type to behave in the same manner as that of variables of built-in types.
For example: if we want to add two complex numbers, then we need to define member function add() and call it using the following notation.

c3.add (c1, c2) ; or c3 = c1.add(c2) ;

The above statements are obscure as the programmer needs to remember the function's name and how to call it. The problem worsens even more when the program becomes large and complex.

A better alternative for performing the addition of two complex numbers is to overload the '+' operator. So the above statements can be written in an easily readable form as

c3 = c1+c2;

This notation is often clearer and more familiar to programmers.

• Operator overloading is useful in large and complex programs involving multiple objects of different classes. It provides a common interface corresponding to an operator that performs similar operations on various objects. For example: In a program, the + operator can add two-time class objects, two date class objects etc.

time1 = time2 + time3;

date1 = date2 + date3;

Operator overloading enhances the capability of C++ language by extending the functionality of most of the existing operators as per your requirement and thus contribute to C++'s extensibility.

• Debugging becomes easier as the code becomes more understandable and clearer.

• Operator overloading helps to gain greater control over objects' behaviour in your program (i.e. controlling the lifetime of objects). For example, – You can overload memory allocation and deallocation functions for your classes to specify exactly how memory should be distributed and reclaimed for each new object.

Program Part:

```
#include<iostream>

using namespace std;

class Price{

public:

float price;

void readData(){

cin >> price;

}

bool operator ==(Price &p){

return price == p.price;

}

};

int main(){
```

```
Price p1, p2;
cout << "Enter First Price: ";
p1.readData();
cout << "Enter Second Price: ";
p1.readData();
if( p1 == p2 ){
cout << "\nPrice are Equal";
}else{
cout << "\nPrice are not Equal";
}
return 0;
}
```

The output of above program is

```
Enter First Price: 100
Enter Second Price: 100
Price are Equal
Enter First Price: 100
Enter Second Price: 200
Price are not Equal
```

25. What is the use of constructor and destructor? Write a program for illustrating default constructor, parameterized constructor and copy constructor.

Solution

Constructor is used to initialize an object of the class and assign values to data members corresponding to the class. While destructor is used to deallocate the memory of an object of a class.

Program Part:

Constructor is a member function of class, whose name is same as the class.

A constructor is a special type of member function of a class which initializes objects of a class. In C++, Constructor is automatically called when object(instance of class) is created.

There are three types of constructor:

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor

Now, We will write a program illustrating the all three types of constructor.

```
// Example: Explicit copy constructor

#include <iostream>

using namespace std;

class Sample{

public:

int id;

Sample(){

id = 5;

cout << "Default Constructor Called:" << endl;

}

Sample(int x){

cout << "\n\nParameterized Constructor Called:" << endl;

id = x;

}

Sample(Sample &s){

cout << "\n\nCopy Constructor Called:" << endl;

id = s.id + 5;

}

void display(){

cout << "ID = " << id;
```

```
}
};
int main(){
Sample obj;
obj.display();
Sample obj1(10);
obj1.display();
Sample obj2(obj1);
obj2.display();
return 0;
}
```

The output of above program is

Default Constructor Called:

ID = 5

Parameterized Constructor Called:

ID = 10

Copy Constructor Called:

ID = 15

26. What is inline function? Why it is used? Write a program to illustrate inline function.

Solution

Inline function is a function that is expanded in line when it is called. When the inline function is called whole code of the inline function gets inserted or substituted at the point of inline function call. This substitution is performed by the C++ compiler at compile time.

Inline function is used :

i. When the performance is required.

ii. It is used outside the class so that we can hide the internal implementation of the function.

iii. It doesn't require any stack on which we can push or pop the variable as it doesn't perform function calling.

Program to illustrate inline function :

```
#include<iostream>

using namespace std;

inline float mul(float x, float y){

return(x * y);

}

inline double div(double p, double q){

return(p / q);

}

int main(){

float a = 12.345;

float b = 9.82;

cout << mul(a,b) << endl;

cout << div(a,b) << endl;

return 0;

}
```

The output of above program is

121.228

1.25713

27. [What are the various class access specifiers? How public inheritance differs from private inheritance?]

Solution

In C++, there are three access specifiers:

- public – members are accessible from outside the class
- private – members cannot be accessed (or viewed) from outside the class
- protected – members cannot be accessed from outside the class, however, they can be accessed in inherited classes

Difference between private and public inheritance in C++

Data hiding is one of the important features of Object Oriented Programming which allows preventing the functions of a program to access directly the internal representation of a class type. The access restriction to the class members is specified by the labeled access modifiers: public, private, and protected sections within the class body.

The default access for members and classes is private.

```
class Base {

public:

// public members go here

protected:

// protected members go here

private:

// private members go here

};
```

A public member is accessible from anywhere outside the class but within a program. You can set and get the value of public variables without any member.

A private member variable or function cannot be accessed, or even viewed from outside the class. Only the class and friend functions can access private members.

28. Write a program to implement function template with multiple arguments.

Solution

Program of Function Template with multiple arguments

```
#include <iostream>

using namespace std;

template<class A,class B> void func(A x,B y)

{

cout << "Value->x: " <<x<< endl;

cout << "Value->y: " <<y<< endl;

}
```

```
int main()
{
func(15.2,1.3);
return 0;
}
```

Output:

Value->x: 15.2

Value->y: 1.3

29. Write a program to illustrate the use of seekg() and tellg().

Solution

Program to use seekg() and tellg()

```
// C++ program to Read a record from a File
// using seekg() and tellg()
#include <bits/stdio.h>
using namespace std;
class student {
int id;
char Name[20];
public:
void display(int K);
};
void student::display(int K)
{
fstream fs;
fs.open("student.dat", ios::in | ios::binary);
// using seekg(pos) method
// to place pointer at 7th record
fs.seekg(K * sizeof(student));
```

```
// reading Kth record
fs.read((char*)this, sizeof(student));
// using tellg() to display current position
cout << "Current Position: "
<< "student no: "
<< fs.tellg() / sizeof(student) + 1;
// using seekg()place pointer at end of file
fs.seekg(0, ios::end);
cout << " of "
<< fs.tellg() / sizeof(student)
<< endl;
fs.close();
}
// Driver code
int main()
{
// Record number of the student to be read
int K = 7;
student s;
s.display(K);
return 0;
}
```

Output:

Current Position: student no: 9 of 100

30. How dynamic memory allocation is done using new and delete? Write a program for illustrating use of new and delete.

Solution

You can allocate memory at run time within the heap for the variable of a given type using a special operator in C++ which returns the address of the space allocated. This operator is called new operator.

If you are not in need of dynamically allocated memory anymore, you can use delete operator, which de-allocates memory that was previously allocated by new operator.

new and delete Operators

There is following generic syntax to use new operator to allocate memory dynamically for any data-type.

```
new data-type;
```

Here, data-type could be any built-in data type including an array or any user defined data types include class or structure. Let us start with built-in data types. For example we can define a pointer to type double and then request that the memory be allocated at execution time. We can do this using the new operator with the following statements -

```
double* pvalue = NULL; // Pointer initialized with null

pvalue = new double; // Request memory for the variable
```

The memory may not have been allocated successfully, if the free store had been used up. So it is good practice to check if new operator is returning NULL pointer and take appropriate action as below -

```
double* pvalue = NULL;

if( !(pvalue = new double ))

{

cout << "Error: out of memory." <<endl;

exit(1);

}
```

The malloc() function from C, still exists in C++, but it is recommended to avoid using malloc() function. The main advantage of new over malloc() is that new doesn't just allocate memory, it constructs objects which is prime purpose of C++.

At any point, when you feel a variable that has been dynamically allocated is not anymore required, you can free up the memory that it occupies in the free store with the 'delete' operator as follows

```
delete pvalue; // Release memory pointed to by pvalue
```

Let us put above concepts and form the following example to show how 'new' and 'delete' work

```
#include <iostream>

using namespace std;

int main ()

{

double* pvalue = NULL; // Pointer initialized with null

pvalue = new double; // Request memory for the variable

*pvalue = 29494.99; // Store value at allocated address

cout << "Value of pvalue : " << *pvalue << endl;

delete pvalue; // free up the memory.

return 0;

}
```

If we compile and run above code, this would produce the following result

```
Value of pvalue : 2949
```

31. Explain the concept of user-defined to user-defined data conversion routine located in the destination class.

Solution

User-defined to user-defined data conversion routines, also known as type conversion operators, allow the programmer to define how an instance of one user-defined data type can be converted to another user-defined data type. This can be useful when working with complex data types that have a similar structure but different names or representations.

These conversion routines are located in the destination class, which is the class that the user wants to convert their object to. The routine takes an object of the source class (the class that the user wants to convert from) as an argument and returns an object of the destination class.

Here's an example in C++:

```
class Celsius {
double temp;
public:
Celsius(double t = 0.0) { temp = t; }
operator Fahrenheit() const;
};
class Fahrenheit {
double temp;
public:
Fahrenheit(double t = 0.0) { temp = t; }
operator Celsius() const {
return Celsius((temp - 32.0) * 5.0 / 9.0);
}
};
Celsius::operator Fahrenheit() const {
return Fahrenheit(temp * 9.0 / 5.0 + 32.0);
}
```

In this example, we have two user-defined data types, Celsius and Fahrenheit, which represent temperature values in different scales. The Celsius class has a conversion routine that allows it to be converted to a Fahrenheit object, and the Fahrenheit class has a conversion routine that allows it to be converted to a Celsius object.

Now, suppose we have a Celsius object c and we want to convert it to Fahrenheit. We can simply use the conversion operator we defined:

```
Celsius c(100.0);
Fahrenheit f = c;
```

In this code, the conversion operator operator Fahrenheit() is called on the c object, which returns a Fahrenheit object. The Fahrenheit object is then assigned to the f variable using the copy constructor.

User-defined to user-defined data conversion routines can be a powerful tool for simplifying code and making it more readable. However, they should be used with care, as they can also make code harder to understand if used excessively or inappropriately.

32. Depict the difference between private and public derivation. Explain derived class constructor with suitable program.

Solution

The difference between public and private declaration are

Public	Private
All the class members declared under public will be available to everyone.	The class members declared as private can be accessed only by the functions inside the class.
The data members and member functions declared public can be accessed by other classes too.	Only the member functions or the friend functions are allowed to access the private data members of a class.
The public members of a class can be accessed from anywhere in the program using the direct member access operator (.) with the object of that class.	They are not allowed to be accessed directly by any object or function outside the class.

For multiple inheritance order of constructor call is, the base class's constructors are called in the order of inheritance and then the derived class's constructor.

```
#include <iostream>

using namespace std;

class Parent1

{

public:

Parent1()

{

cout << "Inside first base class" << endl;

}
```

```
};
class Parent2
{
public:
Parent2()
{
cout << "Inside second base class" << endl;
}
};
class Child : public Parent1, public Parent2
{
public:
Child()
{
cout << "Inside child class" << endl;
}
};
int main()
{
Child obj1;
return 0;
}
```

The output of above program is

Inside first base class

Inside second base class

Inside child class

33.Write a member function called reverseit() that reverses a string (an array of character). Use a for loop that swaps the first and last characters, then the second

and next-to last characters and so on. The string should be passed to reverseit() as an argument.

Solution

```
#include<iostream>
#include<string.h>
using namespace std;
void reverseit(char *p){
int j;
int len=strlen(p);
j=len-1;
for(int i = 0; i < len / 2; i++){
char a=p[i];
p[i]=p[j];
p[j]=a;
j--;
}
cout<<"\nReverse Of String :";
puts(p);
}
int main(){
char str[50];
char ch;
cout<<"\nEnter The String : ";
gets(str);
reverseit(str);
return 0;
}
```

The output of above program is

Enter The String : Suresh

Reverse Of String :hseruS

34. What is the principle reason for using default arguments in the function? Explain how missing arguments and default arguments are handled by the function simultaneously?

Solution

A function can be called without specifying all its arguments. But it does not work on any general function. The function declaration must provide default values for those arguments that are not specified. When the arguments are missing from function call , default value will be used for calculation.

Example:

```
#include <iostream>

using namespace std;

int sum(int a, int b = 10, int c = 20);

int main()

{

cout << sum(11) << endl;

cout << sum(11, 12) << endl;

cout << sum(11, 12, 13) << endl;

return 0;

}

int sum(int a, int b, int c)

{

int z;

z = a + b + c;

return z;

}
```

The output of above program is

41

43

36

35. “An overloaded function appears to perform different activities depending the kind of data send to it” Justify the statement with appropriate example.

Solution

In programming, overloading functions allow multiple functions with the same name to be defined with different input parameters. This enables the same function name to perform different actions depending on the type or number of arguments passed to it.

For example, consider a function called “calculate” that takes two arguments – the first argument is an integer and the second argument is a float. The function performs different calculations depending on the data types of the arguments passed to it. If the first argument is an integer and the second argument is a float, the function calculates the product of the two arguments. If the first argument is a float and the second argument is an integer, the function calculates the sum of the two arguments. The code for the function could look like this:

```
float calculate(int num1, float num2){

return num1 * num2;

}

float calculate(float num1, int num2){

return num1 + num2;

}
```

In this example, the “calculate” function is overloaded to perform different calculations depending on the data types of the arguments passed to it. When the first argument is an integer and the second argument is a float, the first version of the function is called, and when the first argument is a float and the second argument is an integer, the second version of the function is called.

By overloading functions, developers can write more concise and readable code that performs multiple actions with the same function name. It also improves the code’s flexibility, as the function can be used with different data types without having to write multiple functions with different names.

36. Explain the default action of the copy constructor. Write a suitable program that demonstrates the technique of overloading the copy constructor.

Solution

A constructor is a special member function used to initialize objects of its class. It will be called automatically when the object is created.

A class with two or more construct functions with the same name but with different parameters or arguments and other data types is called Constructor overloading.

A copy constructor is a method or member function which initialize an object using another object within the same class.

Example:

```
#include <iostream>

using namespace std;

class ABC

{

public:

int x;

// this is parameterized constructor

ABC(int a){

x = a;

}

// this is copy constructor

ABC(ABC &i){

x = i.x;

}

};

int main()

{

// Calling the parameterized constructor.

ABC a1(40);

// Calling the copy constructor.

ABC a2(a1);
```

```
cout << a2.x;

return 0;

}
```

37. Create a real scenario where static data members are useful. Explain with suitable example.

Solution

Static data members are class members that are declared using the static keyword. There is only one copy of the static data member in the class, even if there are many class objects. This is because all the objects share the static data member. The static data member is always initialized to zero when the first class object is created.

The syntax of the static data members is given as follows -

```
static data_type data_member_name;
```

Example:

```
#include <iostream>

#include <string.h>

using namespace std;

class Student

{

private:

int rollNo;

char name[10];

int marks;

public:

static int objectCount;

Student()

{

objectCount++;

}
```

```
void getdata()
{
cout << "Enter roll number: " << endl;
cin >> rollNo;
cout << "Enter name: " << endl;
cin >> name;
cout << "Enter marks: " << endl;
cin >> marks;
}
void putdata()
{
cout << "Roll Number = " << rollNo << endl;
cout << "Name = " << name << endl;
cout << "Marks = " << marks << endl;
cout << endl;
}
};
int Student::objectCount = 0;
int main(void)
{
Student s1;
s1.getdata();
s1.putdata();
Student s2;
s2.getdata();
s2.putdata();
Student s3;
s3.getdata();
```

```
s3.putdata();
cout << "Total objects created = " << Student::objectCount << endl;
return 0;
}
```

The output of above program is

```
Enter roll number: 1
Enter name: Suresh
Enter marks: 70
Roll Number = 1
Name = Suresh
Marks = 70
Enter roll number: 2
Enter name: Julian
Enter marks: 80
Roll Number = 2
Name = Julian
Marks = 80
Enter roll number: 3
Enter name: Rajesh
Enter marks: 90
Roll Number = 3
Name = Rajesh
Marks = 90
Total objects created = 3
```

38. Create a function called swaps() that interchanges the values of the two arguments sent to it (pass these arguments by reference). Make the function into a template, so it can be used with all numerical data types (char, int, float, and so on). Write a main() program to exercise the function with several types.

Solution

```
#include <iostream>
using namespace std;
template <class T>
void swaps(T &a, T &b){
T temp = a;
a = b;
b = temp;
}
int main(){
int x1, y1;
float x2, y2;
cout << "Enter two Integer numbers: ";
cin >> x1;
cin >> y1;
cout << "Enter two Floting numbers: ";
cin >> x2;
cin >> y2;
cout << "\nBefore Swap:" << endl;
cout << "x1 = " << x1 << " y1 = " << y1 << endl;
cout << "x2 = " << x2 << " y2 = " << y2 << endl;
swaps(x1, y1);
swaps(x2, y2);
cout << "\nAfter Swap:" << endl;
cout << "x1 = " << x1 << " y1 = " << y1 << endl;
cout << "x2 = " << x2 << " y2 = " << y2 << endl;
return 0;
}
```

The output of above program is

Enter two Integer numbers: 2

5

Enter two Floting numbers: 2.2

4.4

Before Swap:

x1 = 2 y1 = 5

x2 = 2.2 y2 = 4.4

After Swap:

x1 = 5 y1 = 2

x2 = 4.4 y2 = 2.2

39. Explain how exceptions are used for handling C++ error in a systematic and OOP-oriented way with the design that includes multiple exceptions.

Solution

Exceptions are run-time anomalies or abnormal conditions that a program encounters during its execution. Exception Handling in C++ is a process to handle runtime errors. We perform exception handling so the normal flow of the application can be maintained even after runtime errors. In C++, we use 3 keywords to perform exception handling: try, catch, and throw

- The try statement allows us to define a block of code to be tested for errors while it is being executed.
- The throw keyword throws an exception when a problem is detected, which lets us create a custom error.
- The catch statement allows us to define a block of code to be executed, if an error occurs in the try block.

Syntax to handle multiple exceptions:

```
try
{
//your code
} catch(exceptionName e1){
// catch block
```

```
} catch(exceptionName e2){
// catch block
} catch(exceptionName en){
// catch block
}
```

The example of exception handling is

```
#include <iostream>
using namespace std;
int main(){
int a = 3;
try
{
if (a == 1)
throw a; // throwing integer exception
else if (a == 2)
throw 'A'; // throwing character exception
else if (a == 3)
throw 4.5; // throwing float exception
}
catch (int a){
cout << "\nInteger exception.";
}catch (char ch){
cout << "\nCharacter exception.";
}catch (float f){
cout << "\nFloat exception.";
}
cout << "\nProgram End.";
return 0;
```

}

Output:

Float exception.

Program End

40. What is the use of new and delete operators? Illustrate with example. What are advantages of new malloc.

Solution

The new operator denotes a request for memory allocation on the Free Store. If sufficient memory is available, new operator initializes the memory and returns the address of the newly allocated and initialized memory to the pointer variable.

Synatx:

pointer-variable = new data-type;

Once we no longer need to use a variable that we have declared dynamically, we can deallocate the memory occupied by the variable.

For this, the delete operator is used. It returns the memory to the operating system. This is known as memory deallocation.

Synatx:

delete pointer-variable;

Advantage of new over malloc are

- new does not need the sizeof() operator where as malloc() needs to know the size before memory allocation.
- Operator new can make a call to a constructor where as malloc() cannot.
- new can be overloaded malloc() can never be overloaded.
- new could initialize object while allocating memory to it where as malloc () cannot.

41. How is character I/O different from Binary I/O? Explain with examples.

Solution

The difference between character I/O and binary I/O are

Character I/O	Binary I/O

In text file, text, character, numbers are stored one character per byte i.e. 32667 occupies 5 bytes even though it occupies 2 bytes in memory.	In binary file data is stored in binary format and each data would occupy the same number of bytes on disks as it occupies in memory.
In the text file, the newline character is converted to carriage-return/linefeed before being written to the disk.	In binary file, conversion of newline to carriage-return and linefeed does not take place.
Text files are used to store data more user friendly.	Binary files are used to store data more compactly.
In the text file, a special character whose ASCII value is 26 inserted after the last character to mark the end of file.	In the binary file no such character is present. Files keep track of the end of the file from the number of characters present.
Content written in text files is human readable.	Content written in binary files is not human readable and looks like encrypted content.

An example to open, read and write to a text file

```
#include <fstream>

#include <iostream>

using namespace std;

int main () {

char data[100];

ofstream myfile;

myfile.open("hello.txt");

cout << "Writing to the file" << endl;

cout << "Enter your name: ";

cin.getline(data, 100);

myfile << data << endl;

cout << "Enter your age: "; cin >> data;

cin.ignore();

myfile << data << endl;
```

```
myfile.close();
ifstream infile;
infile.open("hello.txt");
cout << "Reading from a file" << endl;
infile >> data;
cout << data << endl; infile >> data;
cout << data << endl;
infile.close();
return 0;
}
```

An example to read and write to a binary file

```
#include <iostream>
#include <fstream>
using namespace std;
struct Student{
int roll_no;
string name;
};
int main(){
ofstream wf("student.dat", ios::out | ios::binary);
if (!wf){
cout << "Cannot open file!" << endl;
return 1;
}
Student wstu[3];
wstu[0].roll_no = 1;
wstu[0].name = "Suresh";
wstu[1].roll_no = 2;
```

```
wstu[1].name = "Rajesh";
wstu[2].roll_no = 3;
wstu[2].name = "Prince";
for (int i = 0; i < 3; i++)
wf.write((char *)&wstu[i], sizeof(Student));
wf.close();
if (!wf.good()){
cout << "Error occurred while writing!" << endl;
return 1;
}
ifstream rf("student.dat", ios::out | ios::binary);
if (!rf){
cout << "Cannot open binary file!" << endl;
return 1;
}
Student rstu[3];
for (int i = 0; i < 3; i++)
rf.read((char *)&rstu[i], sizeof(Student));
rf.close();
if (!rf.good()){
cout << "Error occurred while reading!" << endl;
return 1;
}
cout << "Student's Details:" << endl;
for (int i = 0; i < 3; i++){
cout << "Roll No: " << wstu[i].roll_no << endl;
cout << "Name: " << wstu[i].name << endl;
cout << endl;
```

```
}
return 0;
}
```

42. What is this pointer? How can we use it for name conflict resolution? Illustrate with example.

Solution

this is a keyword that refers to the current instance of the class. There can be 3 main usage of this keyword in C++.

- used to pass current object as a parameter to another method.
- can be used to refer current class instance variable.
- used to declare indexers.

One of the main usage of this pointer is to remove name conflict resolution. Let's see an example

```
#include <iostream>
using namespace std;
class Distance{
private:
float inch;
public:
void setValue(float inch){
this->inch = inch;
}
void display(){
cout << "Distance: " << inch;
}
};
int main(){
Distance d1;
d1.setValue(10);
```

```
d1.display();

return 0;

}
```

In the above example, We have on class Distance which has member variable inch. We are going to set value on this.

```
void setValue(float inch){

this->inch = inch;

}
```

If you closely see setValue function then we have passed inch as parameter and we are setting value also on inch.

On this case, compiler get confused because passed parameter and set variable name are same. To fix this issue, We will use this pointer.

43. How can you define catch statement that can catch any type of exception? Illustrate the use of multiple catch statement with example.

Solution

Exceptions are run-time anomalies or abnormal conditions that a program encounters during its execution.

Exception Handling is one of the main advantage of c++.

We will use try and catch block to catch exception. The syntax of try and catch statement is

```
try {

// Block of code to try

throw exception; // Throw an exception when a problem arise

}catch () {

// Block of code to handle errors

}
```

We can catch all exception using three dots ... on catch as parameter. The syntax is given below

```
try {

// Block of code to try
```

```
throw exception; // Throw an exception when a problem arise
}catch (...) {
// Block of code to handle errors
}
```

We can use catch multiple times to catch multiple types of exception. In below example, We have used multiple catch to catch multiple exception.

```
#include <iostream>
using namespace std;
int main()
{
try {
throw 10;
}
//catching character exception
catch (char *c) {
cout << "Caught " << c;
}
//catching all exception
catch (...){
cout << "Exception Occured\n";
}
return 0;
}
```

44. Which functions can be used for reading and writing object? Describe briefly. Write a program that read values of two objects of student class(assume data members are sid , sname, and level) and display the data in monitor.

Solution

Reading and writing data to and from files requires another standard library of C++ <fstream>. The three main data types of fstream are -

- ifstream - represents input file stream and reads information from files.
- ofstream - represents output file stream and writes information to files.
- fstream - represents general file stream and has capabilities of both.

To read or write object from/to file, We will use following functions

file1.write((char*)&Emp_1, sizeof(Emp1));

- Here data present in class object Emp_1 is written to file Employee.txt by calling write function. (char*)&Emp_1 is used to point at the start of an object and sizeof(Emp_1) calculates the number of bytes copied in file.

file2.read((char*)&Emp_1, sizeof(Emp1));

- Here data present in class object Emp_1 is read from file Employee.txt by calling read function. (char*)&Emp_1 is used to point at the start of an object and sizeof(Emp_1) calculates the number of bytes read from the file.

Program Part:

```
using namespace std;

class Student{

public:

int sid;

string sname;

string level;

void read_data(){

cout << "Enter Student ID: ";

cin >> sid;

cout << "Enter Student Name: ";

cin >> sname;

cout << "Enter Student Level: ";

cin >> level;

}

void print_data(){

cout << "\nStudent ID: " << sid << endl;
```

```
cout << "Student Name: " << sname << endl;

cout << "Student Level: " << level << endl;

}

};

int main()

{

Student s1;

s1.read_data();

s1.print_data();

return 0;

}
```

45. What is meant by type conversion? Define two way of converting one user defined data type (object) to another user defined object? Write a program that converts object of another distance class with data members feet and inch.(Assume 1m = 3.3 feet and 1cm = 0.4 inch)

Solution

Type casting refers to changing an variable of one data type into another.

Type conversion can be classified into the following two types:

1.Implicit Type Conversion

When the type conversion is performed automatically by the compiler without programmers intervention, such type of conversion is known as implicit type conversion or type promotion.

```
#include <iostream>

using namespace std;

int main(){

int x = 10; // integer x

char y = 'a'; // character c

// y implicitly converted to int. ASCII

// value of 'a' is 97
```

```
x = x + y;

// x is implicitly converted to float

float z = x + 1.0;

cout << "x = " << x << endl

<< "y = " << y << endl

<< "z = " << z << endl;

return 0;

}
```

2.Explicit Type Conversion

The type conversion performed by the programmer by posing the data type of the expression of specific type is known as explicit type conversion. The explicit type conversion is also known as type casting.

Type casting in c is done in the following form:

(data_type)expression;

Example:

```
#include <iostream>

using namespace std;

int main(){

double x = 1.2;

// Explicit conversion from double to int

int sum = (int)x + 1;

cout << "Sum = " << sum;

return 0;

}
```

Program Part:

```
#include<iostream>

using namespace std;

class Distance{
```

```
public:
int inch = 20;
int feet = 15;
void display(){
cout << "Inch: " << inch << endl;
cout << "Feet: " << feet << endl;
}
};
class Distance_two{
public:
float meter;
float cm;
void operator = (Distance a){
meter = (1 / 3.3) * a.feet;
cm = (1 / 0.4) * a.inch;
}
void display(){
cout << "Meter: " << meter << endl;
cout << "Centimeter: " << cm << endl;
}
};
int main(){
Distance a;
Distance_two b;
//CLass type conversion with operator overloading
b = a;
//displaying data
a.display();
```

```
b.display();

return 0;

}
```

The output of above program is:

Inch: 20

Feet: 15

Meter: 4.54545

Centimeter: 50

46. How ambiguity arises in multipath inheritance? How can you remove this type of ambiguity? Explain with suitable example.

Solution

Multipath Inheritance is a derivation of a class from other derived classes, which are derived from the same base class. In this type of inheritance, there involves other inheritance like multiple, multilevel, hierarchical etc.

There is ambiguity in this type of inheritance.

Ambiguity in Multipath Inheritance:

Suppose we have classes A, B, C and D. classes B and C are derived from A and class D is derived from classes B and C.

Both derived classes inherits the features of base class. Hence when we derive a new class by inheriting features form these two classes derived from the same base class, then same features from the first base is inherited to the finally derived class from two paths. This cause ambiguity in accessing first base class members.

```
# include<iostream>

class A

{

private:

//...

public:

void display(){

//...
```

```
}
};
class B: public A
{
//...
};
class C: public A
{
//..
};
class D: public B, public C
{
//...
};
int main(){
D obj;
obj.display();
return 0;
}
```

To eliminate this problem, C++ has a mechanism to inherit a single copy of properties from the common base class.

This is done by declaring the base class as virtual while creating derive classes from this base class.

After removing ambiguity :-

```
# include<iostream>
class A
{
private:
```

```
//...
public:
void display(){
//...
}
};
class B: virtual public A
{
//...
};
class C: virtual public A
{
//..
};
class D: public B, public C
{
//...
};
int main(){
D obj;
obj.display();
return 0;
}
```

47. What is structured programming? Discuss characteristics and problems associated with structured programming.

Solution

Structure Programming is a programming language that contain well-structured steps and procedures that uses different functions for different tasks in a program.

Characteristics of Structure Programming Language:

i. Emphasis is on doing thing (algorithms)

ii. Large programs are divided into smaller program known as functions

iii. Most of the functions share global data.

iv. Employs top-down approach in program design

v. Data move openly around the system from function to function.

vi. Functions transform data from one form to another

Problems of using Structure Programming

1. Since it is Machine-Independent, So it takes time to convert into machine code.
2. The converted machine code is not the same as for assembly language.
3. The program depends upon changeable factors like data-types. Therefore it needs to be updated with the need on the go.
4. Usually the development in this approach takes longer time as it is language-dependent. Whereas in the case of assembly language, the development takes lesser time as it is fixed for the machine.

48. What is the use of get and getline functions? Explain with suitable example.

Solution

get() is used for accessing character array. It includes white space characters.

Synatx:

get(string_name, size);

Example:

```
#include <iostream>

#include <string.h>

using namespace std;

int main()

{

char name[25];
```

```
//Example Of get
cout << "Enter Your Name: ";
cin.get(name, 25);
cout << "Hello, " << name;
return 0;
}
```

The output of above program is

Enter Your Name: Hamro CSIT

Hello, Hamro CSIT

getline() is a standard library function that is used to read a string or a line from an input stream. The getline() function extracts characters from the input stream and appends it to the string object until the delimiting character is encountered. While doing so the previously stored value in the string object *str* will be replaced by the input string if any.

getline(istream& is, string& str);

Example:

```
#include <iostream>
#include <string.h>
using namespace std;
int main()
{
string name;
//Example Of get
cout << "Enter Your Name: ";
getline(cin, name);
cout << "Hello, " << name;
return 0;
}
```

The output of above program is

Enter Your Name: Hamro CSIT

Hello, Hamro CSIT

49. What is meant by pass by reference? How can we pass arguments by reference by using reference variable? Illustrate with example.

Solution

Pass-by-reference means to pass the reference of an argument in the calling function to the corresponding formal parameter of the called function. The called function can modify the value of the argument by using its reference passed in.

To pass a value by reference, argument pointers are passed to the functions just like any other value. So accordingly you need to declare the function parameters as pointer types as in the following function swap(), which exchanges the values of the two integer variables pointed to, by their arguments.

```
#include <stdio.h>

void swap(int *x, int *y) {

int temp;

temp = *x;

*x = *y;

*y = temp;

return;

}

int main () {

int a = 100;

int b = 200;

printf("Before swap, value of a : %d\n", a );

printf("Before swap, value of b : %d\n", b );

swap(&a, &b);

printf("After swap, value of a : %d\n", a );

printf("After swap, value of b : %d\n", b );

return 0;
```

}

c++ code:

```
#include<iostream>
using namespace std;
void swap(int *x, int *y) {
int temp;
temp = *x;
*x = *y;
*y = temp;
return;
}
int main () {
int a = 100;
int b = 200;
cout << "Before swap, value of a : " << a << endl;
cout << "Before swap, value of b : " << b << endl;
swap(&a, &b);
cout << "After swap, value of a : " << a << endl;
cout << "After swap, value of b : " << b << endl;
return 0;
}
```

The output of above programs is

Before swap, value of a : 100

Before swap, value of b : 200

After swap, value of a : 200

After swap, value of b : 100

It shows that the change has reflected outside the function as well, unlike call by value where the changes do not reflect outside the function.

50. Write short notes on:

a. Manipulators

b. Protected Access Specifier

Solution

a) Manipulator:

Manipulators are operators that are used to format the data display. The most commonly used manipulator are endl and setw.

1) Endl:

The endl manipulator works same as the "\n" (i.e. new line). The endl manipulator, when ysed in an output statement, causes a line feed to be inserted.

Example:

```
cout << "m = 1" << endl;

cout << "n = 2" << endl;
```

The endline in above example will cause to print m = 1 in one line and n = 2 in next line.

2. setw:

The setw manipulator helps to shift the output statements to right. It is used for shifting output right wards according to need.

Example:

```
cout << setw(5) << sum << endl;
```

The manipulator setw(5) in above example will right shift the output value of sum in the screen.

b) Protected Access Specifier

Protected access modifier is similar to private access modifier in the sense that it can't be accessed outside of it's class unless with the help of friend class, the difference is that the class members declared as Protected can be accessed by any subclass(derived class) of that class as well.

Let's take an example

```
#include<iostream>

using namespace std;
```

```
class Parent{
protected:
int protectedID;
};
class Child : public Parent{
public:
void setId(int id){
protectedID = id;
}
void displayId(){
cout << "Protected ID is: " << protectedID << endl;
}
};
int main() {
Child obj1;
obj1.setId(81);
obj1.displayId();
return 0;
}
```

The output of above program is

Protected ID is: 81

In the Base class, We have defined protectedID as private that and Child class also can access protected data of the patrent.

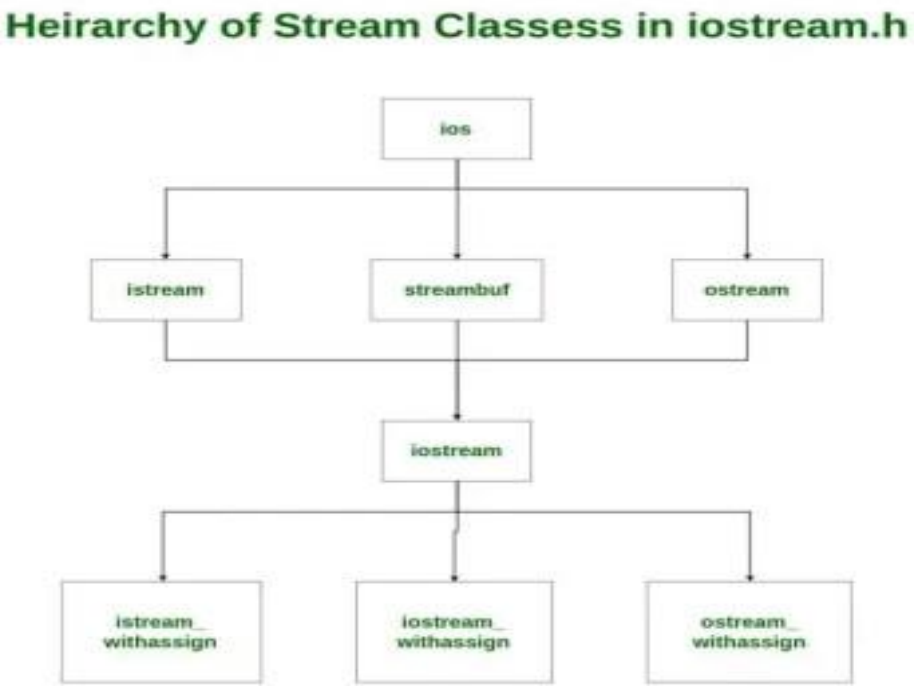
51. Briefly explain the hierarchy of stream classes. Write a program that overloads extraction and insertion operators.

Solution

In C++ stream refers to the stream of characters that are transferred between the program thread and i/o.

Stream classes in C++ are used to input and output operations on files and io devices. These classes have specific features and to handle input and output of the program.

The iostream.h library holds all the stream classes in the C++ programming language.



1. ios class is topmost class in the stream classes hierarchy. It is the base class for istream, ostream, and streambuf class.
2. istream and ostream serves the base classes for iostream class. The class istream is used for input and ostream for the output.
3. Class ios is indirectly inherited to iostream class using istream and ostream. To avoid the duplicity of data and member functions of ios class, it is declared as virtual base class when inheriting in istream and ostream.

<u>Program Part:</u>

Below is the program that overwrite the extraction and insertion operator

```
#include <iostream>

using namespace std;

class Complex

{

private:

int real, imag;
```

```
public:
Complex(int r = 0, int i = 0)
{
real = r;
imag = i;
}
friend ostream &operator<<(ostream &out, const Complex &c){
out << c.real;
out << "+i" << c.imag << endl;
return out;
}
friend istream &operator>>(istream &in, Complex &c){
cout << "Enter Real Part ";
in >> c.real;
cout << "Enter Imaginary Part ";
in >> c.imag;
return in;
}
};
int main()
{
Complex c1;
cin >> c1;
cout << "The complex object is ";
cout << c1;
return 0;
}
```

The output of above program is

Enter Real Part 1

Enter Imaginary Part 2

The complex object is 1+i2